
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON GATE - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

OPERATION IRON GATE

*** FROSTLINE WHITEOUT BUILD RECOVERED ***

TARGET: NorthPharma logistics annex access gate
ARTIFACT: ACT-II.bin / ACT-II.uf2 (compromised)
CREW: FROSTLINE OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma runs the cold chain that keeps a vaccine lot viable between the depot and the clinic. Every annex is protected by an Access Gate and Vault Control node built on a Raspberry Pi Pico 2. The node reads a DHT11 vault interlock, drives a 1602 I2C LCD access log, moves a deadbolt on an SG90 servo, lights a tri-color annunciator (red DENIED, yellow PENDING, green GRANTED), takes a PIN on a VS1838B infrared keypad, senses a request-to-exit button, and exchanges an authenticated authorization and audit uplink over an RYLR998 LoRa radio.

A sabotage crew called **FROSTLINE** poisoned the gate image at the annex. Their implant, **WHITEOUT**, leaves four defects in the compiled firmware. The primitives are standard and correctly implemented: the gate seals every request and every grant with XChaCha20-Poly1305 under an Argon2id field key, and the cipher is exactly what it claims to be. The boundary is the key material: the field passphrase and salt are compiled into the image in cleartext, so anyone holding the `.bin` can re-derive the field key. The crypto is lab-only and is not a secrecy guarantee. The defects are all in the stateful policy around the cipher: a grant can be replayed, a forged grant can be accepted, a debugger can flip the verdict in plain SRAM, and an authorization timeout can fail open. Operative **NIGHTINGALE** pulled the exact compromised image off the annex gate and then went quiet.

You are the reverse-engineering reserve. You get `ACT-II.bin`, a breadboard, and a debug probe. There is no source. Find all four defects, patch the image, demonstrate the SRAM verdict attack under GDB, export a corrected image, and prove on real hardware that the bolt stays shut when it should.

The operation is codenamed **IRON GATE**. If the door lies about who came through, the evidence that ends FROSTLINE is erased with the site.

Scenario Briefing

The frame recovered in Act I led to a NorthPharma logistics annex with one entrance and no windows. NIGHTINGALE's last transmission came from inside it. FROSTLINE is scrubbing the site, and the gate is the last thing standing between the evidence and a bulldozer.

The gate is a decision. Someone types a PIN on the infrared keypad, the gate seals a request to the security desk over LoRa, the desk authenticates the request and answers with a sealed grant, and the bolt moves only if the grant is fresh, authentic, and matches the authorization state the desk intended to store. Two separate questions hide inside that decision. **Authentication** asks who this is. **Authorization** asks whether they are allowed, right now, in this state, to pass. The WHITEOUT image collapses the two, and the door trusts the wrong thing.

The compromise is not in the cipher. It is in four seams around it:

1. **Replay.** A valid GRANTED frame is captured off the air and sent again. The tag is correct, so cryptography is happy. Only a monotonic sequence window can tell the gate that the grant was already used.
2. **Forge.** The tag-difference branch in the AEAD open routine is inverted, so a forged grant with a non-zero Poly1305 difference authenticates while a genuine grant is rejected.
3. **State.** The wire is authenticated and the verdict is not. The gate stores its decision in a plain SRAM boolean, `granted`, and a debug probe can write it. The authenticated-state tag check that should catch the tamper is bypassed.
4. **Fail-open.** When the desk reply never arrives, the correct policy is to stay locked. The compromised policy unlocks on the authorization timeout.

There is also a fifth path that is **not** a defect. Request-to-exit (REX) on GP15 is deliberately unauthenticated for life safety: a person inside the vault must always be able to leave, even in a fire, even if the desk is down, even if every key is lost. The correct response is to name and defend that trade-off, not to bolt it shut.

AUTHORIZED LAB ONLY: This challenge uses a supplied Pico 2 training node and its exact compromised firmware image. Do not connect this exercise to a public network, an operational access-control system, a pharmaceutical network, or any device you do not own or have explicit written authorization to test.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the recurring monitor loop.
- Distinguish authentication from authorization, and explain why a sealed frame alone does not prove freshness or state integrity.
- Analyze an unsigned compare and a conditional-branch byte, and restore a monotonic anti-replay window from an inverted branch.
- Analyze a constant-time Poly1305 tag compare and explain why one inverted branch authenticates forged frames.
- Analyze an authenticated-state gate and explain the TOCTOU failure where the wire is sealed but the verdict is not.
- Analyze a fail-mode policy branch and choose fail-secure over fail-open for an authorization timeout.
- Demonstrate the SRAM verdict attack with GDB and prove that a tampered `granted` boolean fails the state tag before the bolt moves.
- Export and UF2-convert a corrected image and prove the corrected behavior on real hardware.

What This Project Tests

Block	Concepts Tested
1	RP2350 architecture, ARM Cortex-M33 registers, stack, flash/SRAM, Thumb assembly, Ghidra static analysis
2	GDB connection, breakpoints, memory inspection, SWD debugging, serial console observation
3	Bootrom handoff, vector table, reset handler, startup code, XIP, Thumb-bit addressing
4	Function boundaries, call graphs, module mapping, literal pools

Block	Concepts Tested
5	Unsigned compare semantics, condition codes, branch inversion, control flow
6	Anti-replay sequence windows, authenticated-state tags, TOCTOU and state integrity
7	Argon2id memory-hard KDF, XChaCha20-Poly1305 AEAD, Poly1305 tag verification, constant-time comparison
8	Fail-secure versus fail-open policy, designed egress, incident reporting

Part 1: Understanding the System

Access Gate and Vault Control Hardware

Component	Connection	Purpose
Raspberry Pi Pico 2	RP2350	Runs the compromised WHITEOUT image
DHT11 sensor	Data on GPIO 4	Vault environmental interlock
1602 I2C LCD	SDA GPIO 2, SCL GPIO 3, address <code>0x27</code>	Access log
RYLR998 radio	RX GPIO 8, TX GPIO 9, UART1	Authorization and audit uplink
IR receiver	GPIO 5	VS1838B NEC badge reader and PIN keypad
SG90 servo	GPIO 14	Deadbolt actuator, 50 Hz PWM
Red LED	GPIO 16	DENIED
Yellow LED	GPIO 17	PENDING
Green LED	GPIO 18	GRANTED
Request-to-exit button	GPIO 15, internal pull-up	Life-safety egress, intentionally unauthenticated
Onboard LED	GPIO 25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	Authorized GDB inspection

Every graded finding lives in flash (`.text` / `.rodata` / data image) or in SRAM, and is reachable with only the toolset: Ghidra, GDB, and a serial console.

Console and Radio Configuration

- USB-CDC virtual COM port: `115200` baud, `8` data bits, no parity, `1` stop.
- Radio link to the desk: UART1 at `115200`, network identifier `18`.
- Logic level: `3.3 V` only. Never connect `5 V` to a Pico GPIO.

Normal (Intended) Behavior

An honest gate makes a deliberate decision and leaves a record:

Intended Gate Behavior

1. Boot and initialize the keypad, LCD, radio, and actuators
2. Derive the field key with Argon2id
3. Seal a PIN request with XChaCha20-Poly1305 and send it
4. Show yellow PENDING while the desk decides
5. Reject a grant whose seq is not strictly greater than last
6. Accept a grant only when the Poly1305 tag difference is 0
7. Recompute the state tag over the authorization record
8. Move the bolt only when auth_state_ok and the interlock pass
9. Stay locked on an authorization timeout (fail secure)
10. Log the badge, the sequence, and the verdict on the LCD

Observed (Compromised) Behavior

When the WHITEOUT image runs, the gate and the access log disagree with the truth:

Observation	Honest meaning	WHITEOUT behavior
Captured grant, second use	replay must be rejected	reopens the door
Forged grant	tag must reject it	authenticates and moves the bolt
Debugger sets <code>granted = 1</code>	state tag must reject it	moves the bolt anyway
Desk reply never arrives	stay locked	unlocks on the timeout
Replay branch	<code>bcc.n</code> , accept when <code>last_seq < seq</code>	<code>bcs.n</code> , accepts an old sequence
Tag branch	<code>beq.n</code> , accept on zero difference	<code>bne.n</code> , accepts a non-zero difference

Do not assume the first readable status is the truth. Treat every displayed line as evidence to be checked against the machine code.

Part 2: The Firmware

There is no source. FROSTLINE built the WHITEOUT image from the NorthPharma reference firmware and changed **four bytes**. Your job is to reverse engineer `ACT-II.bin` with Ghidra, find every defect, patch the image directly, and prove the corrected behavior on the hardware.

Module Map

The image is stripped. Use these anchor functions and addresses (from the corrected reference image) to orient yourself:

Module	Anchor function	Address
Entry	<code>main</code>	<code>0x10000234</code>
State machine	<code>monitor_init</code>	<code>0x10006858</code>
Desk receive path	<code>monitor_rx_tick</code>	<code>0x10006704</code>
State machine loop	<code>monitor_step</code>	<code>0x100069D4</code>

Module	Anchor function	Address
Interlock	sensor_read	0x10007080
Radio	radio_send_frame	0x100077C0
Annunciator	status_led_show	0x10007ABC
Deadbolt	servo_lock	0x10007BC0
Deadbolt	servo_unlock	0x10007BDC
Infrared keypad	ir_remote_poll	0x10007CA4
Keypad	keypad_push_digit	0x10007D38
Keypad	keypad_clear	0x10007D6C
Keypad	keypad_enter	0x10007D80
Keypad	keypad_get_pin	0x10007DA0
Authorization	auth_begin_request	0x10007E68
Authorization	auth_state_ok	0x10007E80
Authorization	auth_apply_grant	0x10007F2C
Crypto	crypto_aead_open	0x1000809C
KDF	crypto_kdf_argon2id	0x10008150
Envelope	envelope_seal_hex	0x100081C8
Envelope	envelope_open_hex	0x10008288
Random source	get_rand_32	0x100066FC

Annotated disassembly for the key functions is provided in `ACT-II-main-disasm.txt`. Use it as a map, then confirm every byte yourself.

What The Firmware Does

1. Initializes USB-CDC stdio, proves the I2C bus, and configures the keypad, LCD, radio, LEDs, REX button, servo, and infrared receiver.
2. Derives the 32-byte field key with Argon2id from a committed passphrase and salt into the SRAM buffer `g_key` at `0x20013378`.
3. On a latched PIN, seals an unlock request and sends it to the security desk, then shows PENDING and starts the authorization wait.
4. Drains inbound `+RCV` lines, opens the sealed grant, verifies the anti-replay window and the state tag, checks the interlock, and moves the deadbolt.
5. Services request-to-exit and the infrared keypad.
6. On an authorization timeout, applies the configured fail mode.

Defect Summary: What You Are Graded On

Bug #	Name	Severity	Description	Hint
Bug #1	Replay	CRITICAL	The anti-replay branch is inverted, so a captured grant whose sequence is not strictly greater than <code>last_seq</code> reopens the door.	Find the unsigned compare in <code>auth_apply_grant</code> .
Bug #2	Forge	CRITICAL	The Poly1305 tag-difference branch is inverted, so a forged grant with a non-zero difference authenticates.	The correct branch accepts only a zero difference.
Bug #3	State	CRITICAL	The authenticated-state gate is bypassed, so a debugger that sets <code>granted = 1</code> without recomputing the state tag opens the door.	Look inside the inlined <code>monitor_grant_release</code> .
Bug #4	Fail-open	HIGH	The fail-mode branch is inverted, so an authorization timeout unlocks the bolt instead of leaving it locked.	The correct policy is fail secure.

All four defects are same-size in-place byte patches, so no address moves.

The Cryptographic Core Is Real

The crypto core is a correct reference construction. Argon2id derives the field key, XChaCha20-Poly1305 seals every frame, and the Poly1305 tag compare is a correct constant-time word-wise OR fold. Only the four policy seams were broken. Once those bytes are restored, the authenticated envelope verifies as intended against anyone who sees only the wire. That is not a secrecy guarantee: the field passphrase and salt are embedded in the image, the field key is recoverable by anyone holding the `.bin`, and Argon2id at `m=64` KiB is below current memory-hardness guidance. Describe the construction honestly in your report.

Part 3: Your Assignment

Whenever a task asks you to **Document** or **answer**, write your answers in a single file named `ACT-II-Answers.md`. Capture screenshots and terminal transcripts as evidence and reference them from your answers.

Task 1: Setup and Initial Analysis (10 points)

1. Create a new Ghidra project named `IronGate_Investigation`.
2. Import `ACT-II.bin` as a **Raw Binary**.
3. In the language search box type `Cortex`, then select **ARM Cortex 32 little endian default**.
4. Set the base address to `0x10000000`.
5. Run auto-analysis.

Document:

- A screenshot of the Ghidra **Import Results** or **Program Information** window showing the project name, processor settings, and base address.
- The vector-table base, the initial stack pointer, and the reset handler as stored (note its Thumb bit) versus the actual instruction address.
- The address of `main()` and the address of the recurring monitor loop (`monitor_step`).

- The module map: at least one anchor function for the IR keypad, the auth module, the servo deadbolt, the LEDs, the DHT11 interlock, the LCD access log, the REX button, and the sealed LoRa link.

Task 2: Bug #1 Replay (20 points)

1. In Ghidra, find `auth_apply_grant` (starts at `0x10007F2C`) and locate the anti-replay branch at file offset `0x7F43` (VA `0x10007F43`).
2. Document the instruction and the byte at `0x7F43`, and the `seq > last_seq` rule it is supposed to enforce.
3. Patch the byte so the gate accepts a grant only when its sequence is strictly greater than `last_seq`.
4. Confirm that a captured GRANTED frame then fails to reopen the door.

Questions to answer:

- Which byte encodes the condition code, and what do `bcc.n` and `bcs.n` each test after the `cmp last_seq, seq`?
- Why does a strictly monotonic sequence window defeat a replay even when the frame is perfectly authentic?

Task 3: Bug #2 Forge (20 points)

1. In `crypto_aead_open` (starts at `0x1000809C`), locate the tag-difference branch at file offset `0x811D` (VA `0x1000811D`).
2. Document the constant-time compare and the exact branch condition.
3. Patch the byte so a frame is accepted only when the Poly1305 tag difference is zero.
4. Confirm that a forged grant now fails authentication.

Questions to answer:

- Why does accepting a non-zero tag difference break authenticity?
- Why is a constant-time compare used instead of an early-exit byte compare?

Task 4: Bug #3 State (20 points)

1. The `monitor_grant_release` path is inlined into `monitor_rx_tick` (starts at `0x10006704`). Locate the authenticated-state branch at file offset `0x6797` (VA `0x10006797`).
2. Document the `auth_state_ok` gate and the exact branch condition.
3. Patch the byte so the bolt moves only when `auth_state_ok` returns true.
4. Demonstrate with GDB that writing `g_auth.granted = 1` without recomputing the state tag no longer opens the door on the corrected image, while the compromised build opens it.

Questions to answer:

- What is the TOCTOU failure here, stated in terms of the wire and the verdict?
- Why does a keyed tag over the authorization record catch a debugger write that a bare `if (granted)` never would?

Task 5: Bug #4 Fail-Open (20 points)

1. The `monitor_fail_mode` path is inlined into `monitor_step` (starts at `0x100069D4`). Locate the fail-mode branch at file offset `0x6BCB` (VA `0x10006BCB`).
2. Document the fail-secure versus fail-open policy and the branch condition.
3. Patch the byte so an authorization timeout leaves the bolt locked.
4. Confirm that a grant that never arrives leaves the gate locked.

Questions to answer:

- Why is fail-open the wrong default for a security ingress timeout?
- How does this compare to the deliberately fail-safe REX egress path, and why can the two halves of one door need opposite defaults?

Task 6: Export and Verify (10 points)

1. Export the patched program from Ghidra as `ACT-II_fixed.bin`.
2. Convert it to UF2:

```
python uf2conv.py ACT-II_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-II_fixed.uf2
```

3. Run the machine check and confirm it passes:

```
python scripts/verify_ctf.py
```

4. Flash `ACT-II_fixed.uf2` to the Pico 2 and prove on hardware: a replayed grant is rejected, a forged grant is rejected, a debugger write to the verdict is rejected, and an authorization timeout leaves the bolt locked.
5. Write a short reflection mapping each of the four defects to a real-world access-control failure.

How To Breadboard

Wire the peripherals exactly as follows, then power the Pico 2 over USB.

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 data	DATA	GP4	10 kOhm pull-up to 3.3 V if your module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	220 to 330 ohm to GND

Device	Pin on device	Pico 2 GPIO	Notes
Yellow LED	anode	GP17	220 to 330 ohm to GND
Green LED	anode	GP18	220 to 330 ohm to GND
REX button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use **3.3 V logic** on every GPIO. The only 5 V connection is the LCD backpack supply. The 1000 uF capacitor on the servo rail is required to stop the SG90 current spike from browning out the node.

Flash in BOOTSEL mode (hold BOOT, plug in USB) and copy the UF2 onto the RP2350 mass-storage drive, or use `picotool`.

Memory Map Reference

Region	Address	Purpose
Bootrom	0x00000000	Immutable boot code
Flash/XIP	0x10000000	Vector table, code, rodata, data image
SRAM	0x20000000	Stack and writable state
Authorization record <code>g_auth</code>	0x20013334	Verdict, pending flag, <code>seq</code> , <code>last_seq</code>
Authorization key <code>g_auth_key</code>	0x20013350	State-tag key material
Derived field key <code>g_key</code>	0x20013378	32-byte Argon2id field key
Key-ready flag <code>g_key_ready</code>	0x20013963	True once the field key is installed

The VA of any file offset is the file offset plus `0x10000000`. Every defect is a file offset and a VA that differ by exactly that base.

Submission Format

Submit a folder containing:

- `ACT-II-Answers.md` with all written answers;
- screenshots or terminal transcripts, including the GDB state-tamper session;
- `ACT-II_fixed.bin` and `ACT-II_fixed.uf2`;
- the output of `python scripts/verify_ctf.py`;
- the original image SHA-256.

Success Criteria

You complete the challenge when you can prove all of the following:

- You can explain how the RP2350 reaches the gate code from reset.
- You can find and patch all four defect bytes and show the before/after values.
- You can explain why replay and forgery survive a perfect cipher but not a freshness window and an intact tag check.
- You can explain the state-integrity failure and show under GDB that a flipped verdict boolean fails the state tag before the bolt moves.
- You can explain why fail-secure is the correct policy for an authorization timeout and why REX egress is deliberately different.
- You can export, convert, flash, and prove the corrected behavior on real hardware.
- `python scripts/verify_ctf.py` passes.

Academic Integrity

By submitting this CTF work, you certify that:

1. You used only the supplied training node, image, and lab interface.
2. You did not connect the challenge to a public network, an operational access-control system, a pharmaceutical network, or any third-party device.
3. You understand that embedded reverse engineering and binary patching require explicit authorization in any real-world context.
4. You will report any discovered weakness responsibly to the course instructor.

The world is short on people who can read a stripped image and tell an honest byte from a lie. Treat that responsibility seriously: verify before you patch, patch before you trust, and never confuse a green lamp with an open door.

Reference Material

- ARM Cortex-M33 Technical Reference Manual
- RP2350 datasheet
- GDB documentation
- Ghidra documentation: <https://ghidra-sre.org/>
- Argon2 memory-hard function: <https://www.rfc-editor.org/rfc/rfc9106>
- ChaCha20-Poly1305 AEAD: <https://www.rfc-editor.org/rfc/rfc8439>
- PHC reference Argon2: <https://github.com/P-H-C/phc-winner-argon2>
- Project disassembly: `ACT-II-main-disasm.txt`
- Machine verifier: `scripts/verify_ctf.py`